

CNN Convolution Accelerator Architecture Decision Record

FPGA Hardware Design Decisions & Justifications

Project: Convolution Accelerator for CIFAR-10 Image Classification

Target Platform: Xilinx Zynq-7020 (xc7z020clg484-1) on Digilent ZedBoard

Tool: AMD/Xilinx Vivado 2025.2

HDL Language: VHDL

Date: August 2026

Section 1: Project Overview & Scope

The project implements a hardware CNN convolution accelerator targeting the first convolutional layer (Conv1) of a Convolutional Neural Network trained on CIFAR-10. The accelerator is designed as an AXI4-Lite peripheral for the Zynq-7000 Processing System (PS), enabling the ARM Cortex-A9 to offload the most computationally intensive layer to programmable logic (PL).

Scope: Only Conv1 is offloaded to hardware; subsequent CNN layers (Conv2, Conv3, FC1, FC2) remain in software. This is a deliberate architectural decision: Conv1 represents the bottleneck for inference throughput, and offloading only one layer keeps the RTL design tractable while demonstrating measurable speedup.

Section 2: ADR-001 — Target Platform Selection

Decision: Xilinx Zynq-7020 on Digilent ZedBoard

Context: The accelerator needed an FPGA with an integrated ARM processor for host-side software, sufficient logic resources for parallel MAC units, and affordable availability for academic/competition use.

Rationale:

- Zynq-7020 integrates dual-core ARM Cortex-A9 PS with Artix-7 PL fabric, enabling seamless PS-PL communication via AXI interconnect without requiring external soft processors.
- 85K logic cells, 220 DSP48E1 slices, and 4.9 Mb BRAM are sufficient for a 16-channel, 3×3 kernel convolution engine.
- ZedBoard is widely available, well-documented, and used in the SSCS Egypt Competition context.
- The xc7z020clg484-1 speed grade (-1) is the lowest cost option while meeting 100 MHz target frequency.

Alternatives Considered: Standalone Artix-7 (no integrated ARM, would need MicroBlaze soft core — higher resource overhead); Zynq UltraScale+ (overkill for CIFAR-10 32×32 images; higher cost).

Section 3: ADR-002 — CNN Model Architecture for Hardware

Decision: Grayscale single-channel input, 16 output channels, 3×3 kernels, stride 1, same-mode padding

Context: The CNN must classify CIFAR-10 (32×32 RGB) images while keeping the hardware Conv1 layer simple and resource-efficient.

Rationale:

- **Grayscale Conversion:** RGB→grayscale reduces input channels from 3 to 1, cutting input bandwidth and line buffer memory by 66.7%. For a 32×32 image, this means 1 KB vs 3 KB of line buffer storage. Classification accuracy remains acceptable since shape/texture features dominate color for CIFAR-10 categories.

- **16 Output Channels (K=16):** Balances feature richness with hardware parallelism. 16 channels require 16 parallel MAC units — feasible within Zynq-7020's 220 DSP48E1 slices.
- **3×3 Kernel (N=3):** The most common kernel size in modern CNNs. Requires only 9 MACs per output pixel per channel, keeping throughput high. Odd kernel size ensures symmetric padding.
- **Stride 1:** Preserves spatial resolution (32×32 → 32×32 before pooling). Max pooling is deferred to software.
- **Same-Mode Padding:** Zero-pads input by floor(N/2)=1 pixel on each border, producing padded 34×34 image. Output spatial dimensions match unpadded input (32×32). Padding is applied in software/pre-processing before streaming to hardware.

CNN Full Architecture (for context — only Conv1 is in hardware):

- Conv1: 1→16 channels, 3×3, stride 1, padding 1, ReLU, MaxPool(2×2) → 16×16×16
- Conv2: 16→32 channels, 3×3, padding 1, ReLU, MaxPool(2×2) → 32×8×8 (SW only)
- Conv3: 32→32 channels, 3×3, padding 1, ReLU (SW only)
- Dropout(0.5) (SW only)
- FC1: 2048→256, ReLU (SW only)
- FC2: 256→10, Softmax (SW only)

Training: 50 epochs max, Adam optimizer (lr=1e-3), StepLR (decay 0.5 every 10 epochs), batch size 128, early stopping patience 10, seed 2026 for reproducibility.

Section 4: ADR-003 — Fixed-Point Quantization Strategy

Decision: Per-channel asymmetric quantization with independent weight fraction bits, symmetric int8 weights, pre-scaled int32 bias, and 5-bit configurable right-shift

Context: Floating-point arithmetic is prohibitively expensive on FPGA fabric. A fixed-point scheme must be chosen that preserves accuracy while fitting within DSP48E1 multiplier dimensions.

Rationale & Numeric Format:

Signal	Format	Width	Range	Justification
Input Pixel	Unsigned Q0.8	8-bit	[0, 255]	Direct uint8 pixel values; division by 256 implied by fractional interpretation.
Weight Coefficient	Signed int8	8-bit	[-127, +127]	Symmetric range avoids the -128 boundary case. Maps directly to DSP48E1.
Bias	Signed int32	32-bit	Full int32	Pre-scaled to accumulator unit system.
Accumulator	Signed	24-bit	—	Minimum 21 bits needed. Rounded to 24 for byte alignment.
Right-Shift	Unsigned	5-bit	[0, 31]	Per-channel shift factor. Aligns accumulator to Q8.8 output format.
Output	Signed int16	16-bit	[-32768, +32767]	Q8.8 format. Saturating clamp after shift.

Weight Quantization Formula (per channel c):

- $Fw(c) = \max(0, \text{floor}(\log_2(127 / \max(|w_c|))))$

- $q_weight = clip(round(w \times 2^{Fw}), -127, 127)$
- $q_bias = round(bias \times 2^{(Fw + 8)})$
- $shift = Fw$

Rounding: Round-half-up (toward +infinity): $shifted = (acc + 2^{(shift-1)}) \gg shift$. This matches hardware behavior of adding a rounding constant before arithmetic right-shift.

ReLU: Optional per-channel via 1-bit register. When enabled: $output = max(0, saturated_value)$.

Section 5: ADR-004 — Accumulator Bit-Width Sizing

Decision: 24-bit signed accumulator (CFG_ACCUM_WIDTH = 24)

Context: The accumulator must hold the maximum possible sum-of-products plus bias without overflow.

Analysis:

- Minimum width = $W_pixel + W_weight + ceil(log_2(N^2)) + 1 = 8 + 8 + ceil(log_2(9)) + 1 = 8 + 8 + 4 + 1 = 21$ bits
- With 32-bit bias pre-loaded, the accumulator effectively operates at 32-bit precision during bias addition.

Rationale for 24 bits:

- 21 bits is the absolute minimum; 24 bits provides 3 bits of headroom for potential kernel size increases (up to $N=7$ without modification).
- 24 bits aligns to 3-byte boundaries, simplifying memory packing.
- DSP48E1 slices have 48-bit internal accumulators — 24 bits uses only half the available precision, leaving room for chaining.
- The bias (32-bit) is pre-loaded as the accumulator initial value; the 9 MAC operations add at most $8+8+4=20$ bits of range, which fits within 32 bits.

Section 6: ADR-005 — AXI4-Lite Control Interface

Decision: AXI4-Lite slave interface with 32-bit data/address bus for processor-to-accelerator configuration

Context: The host ARM processor must configure 144 kernel weights, 16 biases, 16 shift values, and 16 ReLU enable flags before each convolution pass.

Rationale:

- AXI4-Lite is the simplest AXI protocol, requiring minimal logic for a register-mapped peripheral.
- 32-bit data bus matches ARM Cortex-A9 native word size and Zynq PS-PL GP AXI port width.

- No burst capability needed — configuration registers are written once before each inference pass (not performance-critical).
- AXI4-Lite does not require FIFO/buffer management — simpler verification and lower area.

Alternatives Considered: AXI4-Full (burst mode unnecessary for ~200 register writes); AXI4-Stream for pixel data (used separately for datapath, not for configuration).

Section 7: ADR-006 — Register File Memory Map Design

Decision: Channel-strided memory map with 256-byte (64-word) blocks per channel

Memory Map Layout:

- Channel C base address = $C \times 0x100$
- Words 0–8 (offsets 0x00–0x20): Kernel coefficients w0–w8 (8-bit in low byte of 32-bit word)
- Word 32 (offset 0x80): Channel bias (full 32-bit)
- Word 33 (offset 0x84): Packed shift[4:0] and relu_en[8]

Rationale:

- 256-byte stride per channel enables simple address decoding: channel = addr[15:8], word = addr[7:2].
- Large stride leaves room for future expansion (additional registers per channel) without remapping.
- Coefficient sign-extension on readback (8-bit signed → 32-bit signed) enables software verification via AXI reads.
- Packing shift and relu_en into a single register reduces total register count and write transactions.
- Parallel output ports drive datapath combinatorially — zero latency from register write to datapath availability.

Section 8: ADR-007 — Sliding Window Generator Architecture

Decision: Line-buffer-based 2D sliding window with SRL32-inferred shift registers

Context: The convolution engine requires N×N pixel windows from a raster-scan 1D pixel stream. The window generator must produce one valid window per clock cycle at steady state.

Implementation:

- Uses $N-1 = 2$ line buffers, each IMAGE_WIDTH = 34 pixels deep.
- Line buffers are implemented as arrays of std_logic_vector — a coding style that enables Vivado to infer SRL32 (Shift Register LUT) primitives.
- Window registers form a 3×3 shift register matrix fed by line buffer outputs.

- Spatial indexing: $\text{window_out}(r \times N + c) = \text{window_regs}(r)(N-1-c)$, mapping the oldest pixel to the top-left position for correct kernel alignment.

Rationale:

- SRL32 inference avoids consuming BRAM resources (only $2 \times 34 \times 8 = 544$ bits of LUT shift registers needed). BRAMs are preserved for larger frame buffers or FIFOs.
- Single-clock-cycle throughput: one new pixel in $\rightarrow$ one new valid window out (after N-1 row pipeline fill latency).
- Valid output masking: `valid_out` asserted only when `col_cnt >= N-1 AND row_cnt >= N-1`, suppressing invalid partial windows during pipeline startup.
- For 34×34 padded image with 3×3 kernel: produces $32 \times 32 = 1024$ valid windows matching unpadded output dimensions.

Alternatives Considered: BRAM-based line buffers (wastes 18K BRAM tiles for only 34 bytes of storage); external memory line buffers (adds latency and complexity).

Section 9: ADR-008 — Verification & Validation Strategy

Decision: Three-tier verification approach: golden model $\rightarrow$ unit testbenches $\rightarrow$ system comparison

Tier 1: Python Golden Model (Bit-Exact Reference)

- `golden_conv.py` implements the exact hardware arithmetic in Python using arbitrary-precision integers.
- Matches RTL behavior bit-for-bit: round-half-up rounding, int16 saturation, optional ReLU.
- Comparison against PyTorch float32 reference validates quantization error is acceptable.

Tier 2: VHDL Unit Testbenches

- `tb_coeff_bias_shift_regfile`: Tests register storage, readback with sign-extension, packed bitfield decoding, and parallel output port correctness.
- `tb_axi_lite_ctrl`: Tests AXI4-Lite protocol handshakes (write/read), address decoding, and end-to-end register configuration via BFM procedures.
- `tb_window_generator`: Tests 2D sliding window formation with a scaled-down 5×5 image (N=3), verifying spatial alignment of first and last valid windows.

Tier 3: System-Level Comparison

- `generate_test_vectors.py` produces \$readmemh-compatible hex stimulus and golden output files.
- `compare_results.py` performs automated line-by-line hex comparison between RTL simulation output and golden reference.
- Test vectors include: trained model weights, hand-crafted filters (identity, Sobel X/Y, box blur), all-zeros corner case, all-255 corner case.

Rationale: Bottom-up verification from register-level to system-level catches bugs at the earliest possible stage. The golden model serves as a single source of truth shared between hardware and software teams. Automated hex comparison enables CI/CD integration.

Section 10: ADR-009 — HDL Language & Design Style

Decision: VHDL with strongly-typed custom packages

Rationale:

- VHDL's strong type system catches width mismatches and type errors at compile time, reducing simulation debug cycles.
- Custom unconstrained array types (`coeff_array_t`, `bias_array_t`, etc.) enable generic/parameterized entity ports that scale with K and N without code changes.
- Centralized configuration package (`config_pkg.vhd`) ensures all modules use consistent parameters — a single change propagates everywhere.

Alternatives Considered: SystemVerilog (more concise syntax but less strict typing; team expertise favored VHDL).

Section 11: Summary of All Architecture Decisions

ADR	Decision	Key Rationale
ADR-001	Zynq-7020 on ZedBoard	Integrated ARM+FPGA, sufficient resources, competition availability
ADR-002	Grayscale, 16ch, 3×3, stride 1, same-plane	66% bandwidth reduction, DSP48E1 fit, spatial resolution preservation
ADR-003	Per-channel int8/int32/int16 fixed-point	DSP48E1 multiplier compatibility, symmetric weight range, configurable shift
ADR-004	24-bit accumulator	21-bit minimum + 3-bit headroom, byte-aligned, DSP48E1 compatible
ADR-005	AXI4-Lite for configuration	Simplest AXI protocol, matches Zynq GP port, register-mapped
ADR-006	256-byte channel-strided register map	Simple bit-field decoding, expansion room, zero-latency parallel outputs
ADR-007	SRL32-inferred line buffer window generation	Preserves BRAM, single-cycle throughput, correct spatial alignment
ADR-008	Three-tier verification (golden+TB+compiler)	Bit-accurate reference, bottom-up testing, CI-compatible automation
ADR-009	VHDL with typed packages	Strong typing, parameterized generics, centralized configuration